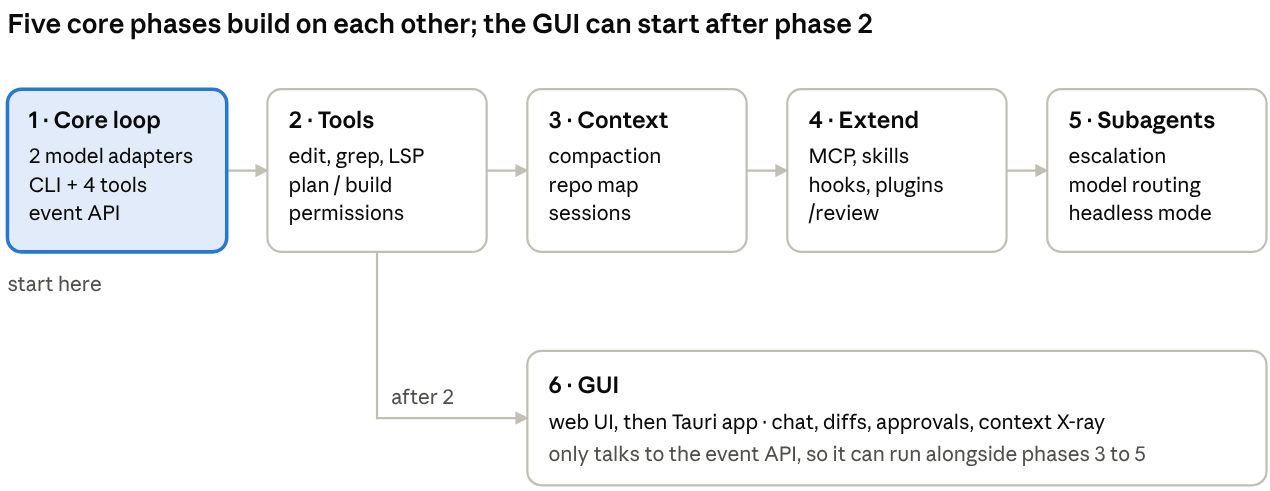
Smok: Harness Plan

Oct 18, 2026 · @Mateusz Pawlowski

Smok (Polish for "dragon") is a model-agnostic coding harness: one agent loop that drives local Ollama models and frontier APIs, with a CLI and a GUI on the same core. Its edge is treating local and frontier models as one team, starting every task locally and escalating only when the local model gets stuck.

Roadmap



roadmap · 6 phases

Each phase ships something usable on its own. The escalation ladder and shadow mode, Smok's signature features, land in phase 5 once routing exists.

Goals

Yes, frontier and local models can run through the same harness. Every major provider uses the same basic shape (messages and tool definitions in, text or tool calls out), so a thin adapter per provider family covers all of them.

The harness is done when it has:

- A CLI and a GUI that run sessions on Claude, GPT, Gemini or any Ollama model, switchable mid-session with `/model`.
- Coding-grade tools: read, edit, search and shell, behind permission prompts.

- Long sessions that hold up: compaction, prompt caching and resumable sessions.
- Extension points: MCP servers, skills, hooks and subagents.
- A system prompt lean enough that a 27B local model can still drive it.

Each phase below ships something usable on its own, so the project never sits half-built.

Brand

The name is Smok, Polish for "dragon", run as `smok` in the terminal. The mascot is a small mischievous dragon, drawn as 32×32 pixel art. Mockups and icons live on the design canvas.

Colour	Hex	Used for
Charcoal	#1A1A1F	Backgrounds, terminal
Ember	#E8562A	Primary, dragon body
Deep ember	#C2410C	Accents, shadows
Bone	#D8BFA3	Horns, secondary
Smoke	#F3ECE6	Text on dark, cursor
Peach	#FDE4D8	Light backgrounds, avatar circle

- **Prompt icon** (the `>_` dragon): app icon, terminal and favicon.
- **Mascot**: chat avatar in the GUI, with expressions as status (working, done, escalated, error).
- **Profile**: README header and website.
- Before publishing, check that `smok` is free on GitHub, PyPI and npm.

Interfaces

The CLI and GUI are both clients of the core's event API, so they stay equal feature for feature. A session started in the terminal can be watched in the GUI.

CLI

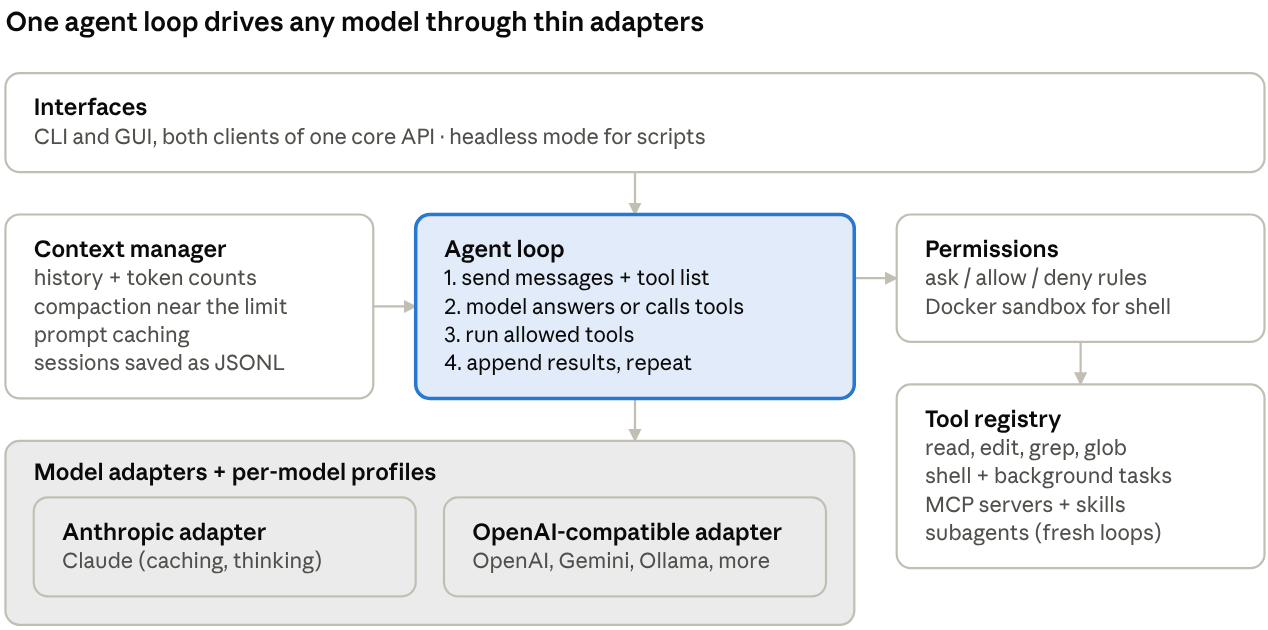
- Status bar: mode, active model (flagged when escalated), context size, cost and checkpoint number.

- Tab toggles plan and build mode; diffs show inline; permission prompts offer yes, always allow and no.
- Slash commands: `/model`, `/context` (context X-ray), `/estimate` (cost preview), `/undo`, `/review`, `/share`.

GUI

- Left: sessions, plus background runs such as model council and shadow mode.
- Centre: chat with collapsible tool calls, an escalation banner, diffs with accept or reject, and permission buttons.
- Right: context bar with pin and drop, the escalation ladder, checkpoints with restore and branch, and session cost.
- Built as a local web UI first (FastAPI, React, shadcn/ui), then wrapped in Tauri as a desktop app.

Architecture



harness architecture · 6 parts

Every provider sits behind one of two adapters, so swapping Claude for a local qwen model changes nothing above the adapter layer.

The core never prints to a screen. It emits events (text, tool call, permission request, diff) over a local API, and the CLI and GUI are two clients of it. That keeps both in sync feature for feature, and lets you start a session in the terminal and watch it in the GUI.

Suggested repo layout:

```
harness/
  cli.py            # entry point, REPL, slash commands
  loop.py           # the agent loop
  models/           # base.py, anthropic.py, openai_compat.py, profiles.yaml
  tools/            # registry.py, files.py, shell.py, search.py
  context.py        # token counts, compaction, caching
  permissions.py
  mcp_client.py
  sessions.py
tests/
evals/
```

Model layer

Write two small adapters of your own rather than binding to one SDK: one for the Anthropic Messages API and one for the OpenAI-compatible format. The second covers OpenAI, Ollama, LM Studio, vLLM, OpenRouter and Gemini's OpenAI-compatible endpoint.

Approach	Upside	Downside
Own adapters (recommended)	Full control of streaming, prompt caching and thinking settings	About 3 files of code to own
LiteLLM as the adapter	Many providers behind one call	Another layer between you and provider features; new options can lag

How it fits together:

1. **Internal types.** A `Message` holds content blocks (text, tool call, tool result, image). A `ToolSpec` is a name, a description and a JSON schema. The loop only ever sees these.

2. **Adapter interface.** `stream(system, messages, tools, params)` yields normalized events: text delta, tool call, usage, stop. Each adapter translates to and from its provider's format.
3. **Model profiles** in a YAML file: context window, tool-calling support, parallel tool calls, image input, max output, cost per million tokens, and prompt style (`lean` or `full`). The harness reads the profile to pick the system prompt size and compaction threshold.
4. **Keys** come from environment variables or the OS keychain, never from a committed config file.

Models without native tool calling (phi4, deepseek-r1) would need a text-based fallback where the model writes JSON and the harness parses it. Leave that out until the core works; stick to tool-capable models like qwen3.6, devstral and gpt-oss.

Phase 1: MVP loop

Goal: chat with tools in a terminal, on any model. Python 3.12 with `uv`, `rich` for output and `prompt_toolkit` for input.

- [ ] Project skeleton and internal message types
- [ ] Anthropic adapter and OpenAI-compatible adapter, both streaming
- [ ] Agent loop with an iteration cap so a confused model can't spin forever
- [ ] Core emits events instead of printing, so the CLI is just the first client and the GUI can plug in later
- [ ] Tools: `read_file`, `write_file`, `list_dir`, `run_shell` (timeout and output cap)
- [ ] Yes/no confirmation before any write or shell command
- [ ] Slash commands: `/model`, `/clear`, `/exit`

Exit check: the same small task in a sample repo succeeds on a Claude model and on qwen3.6 through Ollama.

Phase 2: Coding tools and safety

Goal: good enough to use on your own repos every day. Tool design matters more than anything else here, especially for smaller models.

- [ ] `edit_file` as exact string replace: the match must be unique, and the tool returns a diff

- [ ] `read_file` with line numbers plus offset and limit for big files
- [ ] `grep` (ripgrep) and `glob`
- [ ] Read-before-edit rule: the harness tracks file hashes and refuses edits to files the model hasn't read or that changed since
- [ ] Permission modes (ask, auto-accept edits, full auto) plus allow rules by command pattern, such as always allowing `git status`; saved per project in `.smok/settings.json`
- [ ] Optional Docker sandbox for shell commands, repo mounted, network off by default
- [ ] Checkpoints before each turn (shadow git commits) so `/undo` rolls back the model's changes
- [ ] Load a project instructions file (`AGENTS.md`) into the system prompt

Phase 3: Context and sessions

Goal: long sessions that don't degrade or get expensive. This is where local models live or die.

- [ ] Token counting per model (exact for API models, estimated for local ones)
- [ ] Large tool outputs truncated, with the full text saved to a file the model can read if it needs to
- [ ] Auto-compaction near the context limit: older turns become a structured summary (goal, files touched, decisions, open todos)
- [ ] Prompt caching on Anthropic models for the system prompt, tool list and older history
- [ ] Sessions saved as JSONL, with `--resume` and `--continue`
- [ ] A todo tool so the model keeps its own plan visible across long tasks

Phase 4: Extensibility

Goal: add capabilities without touching the core loop.

- [ ] MCP client using the official Python `mcp` SDK: stdio and HTTP servers from config, tools namespaced by server
- [ ] Skills: folders with a `SKILL.md`; only names and descriptions sit in the prompt, full instructions load on demand
- [ ] Hooks: shell commands before or after tool use (run a formatter after edits, block risky commands)

- [ ] Custom slash commands defined as markdown files
- [ ] Web fetch and web search tools

Phase 5: Subagents and parallel work

Goal: split big tasks so no single context fills up. Subagents help local models most, because each one starts with a small, clean context.

- [ ] Subagent tool: spawns a fresh loop with its own context and a restricted tool set, and returns only a summary
- [ ] Model routing per role: a local or cheap model for search and exploration, a frontier model for planning and edits
- [ ] Parallel subagents with `asyncio`
- [ ] Background shell processes (dev servers, watchers) with output the model can poll
- [ ] Headless mode: `smok -p "task"` returns JSON, so scripts and other tools can call it

Phase 6: GUI

Goal: a full app for people who don't want a terminal. It can start right after Phase 2, since it only talks to the core's event API.

- [ ] Local web UI: FastAPI serves a React and shadcn/ui front end over websockets
- [ ] Chat view with streaming text and collapsible tool calls
- [ ] Diff viewer with accept or reject per change
- [ ] Permission prompts as buttons, with an "always allow" option
- [ ] Model picker, session list and a live cost meter
- [ ] Settings screen for API keys, MCP servers and permission rules, so nobody edits JSON by hand
- [ ] Image input by drag and drop for vision-capable models
- [ ] Desktop app: wrap the web UI in Tauri for a native window and tray icon

Standout features

These are what would make it more than another Claude Code clone. Most fall out of having local and frontier models behind one loop, which few harnesses lean into.

Feature	What it does	Builds on
Escalation ladder	Starts every task on a local model; if tests fail or it loops, hands off to a frontier model with a compact summary of what was tried	Compaction, model routing
Shadow mode	A local model quietly answers alongside the frontier model, and the harness scores how often local would have matched. Over time you learn which tasks local can own	Model routing, evals
Model council	Runs one task on 2 or 3 models in separate git worktrees, then keeps the version that passes tests, or lets a judge model pick	Checkpoints, parallel subagents
Session branching	Fork a session at any past turn to try a different approach, then diff the two outcomes	Checkpoints, saved sessions
Privacy shield	Masks names, emails and secrets before a prompt leaves your machine and restores them in the reply; skipped for local models. A natural first user of your PII masking library	Model adapters
Remote approvals	The harness runs on your homelab, and your phone gets a push notification when it needs a permission or finishes	Headless mode, web UI
Watch mode	Write a comment like `# ai: add retries here` in any file and the harness picks it up and does the work	Coding tools
Replay lab	Re-run a logged real session against a new model or system prompt and compare results, so evals come from your actual usage	Session logs, evals

The escalation ladder and shadow mode pair well: shadow mode tells you where to set the ladder's first rung.

Feature landscape

The plan was missing about a dozen features that other harnesses already ship. The biggest are plan/build modes, LSP code intelligence, a repo map, plugins and a review command. Each is slotted into a phase below.

Feature	Who has it	Lands in
Plan and build modes: read-only planning, toggled with Tab	OpenCode, Cline	Phase 2
A different model per mode: strong planner, cheap editor	Aider, Cline	Phase 5
LSP diagnostics, definitions and references as tools	OpenCode	Phase 2
Repo map: a tree-sitter outline of the codebase, ranked by relevance	Aider	Phase 3
Handoff: start a fresh thread from an editable summary	Amp	Phase 3
Budget and turn caps per run	Claude Code	Phase 3
Self-check before declaring a task finished	Goose	Phase 2
Plugins that bundle skills, hooks and commands	Claude Code, Pi	Phase 4
The agent writes its own extensions and reloads them mid-session	Pi	Phase 4
Reusable workflows with parameters (recipes)	Goose	Phase 4
Built-in code review of uncommitted changes or a branch	Codex	Phase 4
Browser automation to check UI changes	Cline	Phase 4, via a Playwright MCP server
A second-opinion model on demand (an "oracle")	Amp	Phase 5

Feature	Who has it	Lands in
Remote sandboxes running tasks in parallel	Codex, Amp	Phase 5, on homelab runners
Read-only session share links	OpenCode, Amp	Phase 6
Voice input	Aider, Cline	Phase 6

Gaps few harnesses cover well, worth owning:

- **Context X-ray.** A live panel showing everything in the context window by token share (system prompt, tools, files, history), with pin and drop controls. Pi exists largely because its creator found other harnesses' context handling opaque.
- **Per-model tuning.** Each model profile picks its own prompt size, tool set and edit format, and the eval suite tunes them. Most harnesses let you switch models but run them all through the same prompt.
- **Cost preview.** Before a big task, estimate tokens and cost on each configured model, with local shown as $0.
- **Local and frontier as a team.** Many harnesses support local models; the escalation ladder and shadow mode above treat them as one system.

Testing and evals

A small eval suite, run on every change, is how you'll know a prompt or tool tweak actually helped instead of guessing.

- [ ] Unit tests for tools and adapters, using recorded provider responses as fixtures
- [ ] An eval set of 15 to 20 tasks in a fixture repo, each with a check script; record pass rate, turns and tokens per model
- [ ] Every session logged as JSONL for replay and debugging
- [ ] Later: run a slice of a public benchmark (SWE-bench Lite or Terminal-Bench) to compare against existing harnesses

Open decisions and risks

Decision	Options	Leaning
Language	Python or TypeScript	Python: strong MCP SDK and `uv` tooling
Model adapters	Own adapters or LiteLLM	Own adapters, two families
GUI stack	Local web UI, Tauri desktop app, or Electron	Local web UI first, wrapped in Tauri later (lighter than Electron)
Fallback for models without tool calling	Build it or skip it	Skip for now
Name	Smok, Rung or Linka	Decided: Smok

Risks:

- **Local models loop or send malformed tool calls.** Validate arguments against the JSON schema, return clear errors to the model, and keep the iteration cap.
- **Shell commands do damage.** Default to asking, offer the Docker sandbox, keep checkpoints for `/undo`.
- **Scope creep.** Each phase has to ship something usable before the next starts.
- **API costs during evals.** Use prompt caching and run most evals on cheaper or local models.

First steps

- [ ] Check `smok` is free on GitHub, PyPI and npm, then create the repo
- [ ] Scaffold the project with `uv`: core package, CLI entry point, tests folder
- [ ] Write the Anthropic and OpenAI-compatible adapters and run one prompt through each
- [ ] Get the loop working with `read_file` and `run_shell` on qwen3.6 through Ollama
- [ ] Write the first 5 eval tasks, so every change after that is measured

Sources

- OpenCode overview (OpenReplay)
- Cline Plan and Act docs
- Cline overview docs
- Cline (AI Wiki)
- Aider (Codegen)
- Aider (Vantaige)
- Amp: switching guide
- Amp: orbs
- Amp feature notes
- Goose recipes
- Goose v1.41.0 release notes
- Pi: what they didn't build
- Mario Zechner on Pi (AI Engineer)
- Claude Code course (Frontend Masters)
- Claude Code features notes
- Codex CLI commands